LAPORAN

WORKSHOP SISTEM INFORMASI WEB SERVER (TIF330805)

SEMESTER 3



POSTMAN DAN API AUTHENTICATION

Disusun oleh :

Mochammad Syifaul Qulub (E41250219)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Application Programming Interface (API) merupakan mekanisme yang memungkinkan dua aplikasi atau sistem berbeda saling berkomunikasi dan bertukar data. Dalam pengembangan aplikasi berbasis web, API digunakan untuk menghubungkan bagian frontend dengan backend melalui request dan response menggunakan protokol HTTP.

Pengujian API diperlukan untuk memastikan setiap endpoint dapat bekerja sesuai dengan kebutuhan. Pengujian mencakup pemeriksaan metode HTTP, format data JSON, validasi input, status code, serta penanganan kesalahan yang mungkin terjadi selama proses komunikasi.

Pada praktikum Acara 19 dengan pokok bahasan Postman dan API Authentication, dilakukan pengembangan API menggunakan PHP Native dan PDO. Implementasi meliputi pengambilan data mahasiswa, registrasi pengguna, validasi data, serta pengamanan password menggunakan fungsi password_hash(). Pengujian request dilakukan menggunakan PowerShell sebagai alternatif Postman.

1.2 Tujuan Praktikum

Tujuan praktikum Acara 19 adalah sebagai berikut.

1. Memahami konsep pengujian API menggunakan request dan response HTTP.
2. Mengimplementasikan endpoint API menggunakan PHP Native.
3. Mengelola data menggunakan database MySQL dan PDO.
4. Membuat endpoint registrasi pengguna dengan format JSON.
5. Menerapkan validasi input dan validasi format email.
6. Mengamankan password menggunakan metode hashing.
7. Memahami penggunaan HTTP status code dalam penanganan respons API.
8. Menguji keberhasilan dan kegagalan request API menggunakan PowerShell.

BAB 2. LANDASAN TEORI

2.1 Application Programming Interface (API)

Application Programming Interface (API) merupakan antarmuka yang memungkinkan suatu aplikasi berkomunikasi dengan aplikasi atau layanan lain melalui aturan dan format tertentu. API berbasis web umumnya menggunakan protokol HTTP untuk menerima request dan mengirimkan response.

2.2 HTTP Request dan Response

HTTP request merupakan permintaan yang dikirimkan klien kepada server untuk menjalankan suatu operasi. HTTP response merupakan jawaban server terhadap permintaan tersebut.

Request API dapat terdiri atas metode HTTP, URL endpoint, header, dan body. Response API umumnya berisi status code, header, serta body yang memuat hasil pemrosesan server.

2.3 HTTP Method

HTTP method digunakan untuk menentukan jenis operasi yang dilakukan terhadap suatu resource. Beberapa metode yang umum digunakan adalah:

1. GET, untuk mengambil data dari server.
2. POST, untuk mengirimkan data atau membuat data baru.
3. PUT, untuk memperbarui keseluruhan data.
4. PATCH, untuk memperbarui sebagian data.
5. DELETE, untuk menghapus data.

Pada praktikum ini, metode GET digunakan untuk mengambil data mahasiswa, sedangkan POST digunakan untuk melakukan registrasi pengguna.

2.4 JavaScript Object Notation (JSON)

JSON merupakan format pertukaran data yang memiliki struktur sederhana dan mudah dibaca oleh manusia maupun mesin. Format ini sering digunakan dalam

komunikasi API karena dapat merepresentasikan data dalam bentuk objek dan array.

Dalam PHP, fungsi json_encode() digunakan untuk mengubah data PHP menjadi JSON, sedangkan json_decode() digunakan untuk membaca JSON menjadi struktur data PHP.

2.5 HTTP Status Code

HTTP status code merupakan kode yang menunjukkan hasil pemrosesan request oleh server. Status code yang digunakan atau diuji pada praktikum ini meliputi:

1. 200 OK, menunjukkan request berhasil.
2. 201 Created, menunjukkan resource baru berhasil dibuat.
3. 400 Bad Request, menunjukkan request tidak memenuhi ketentuan.
4. 404 Not Found, menunjukkan endpoint atau resource tidak ditemukan.
5. 405 Method Not Allowed, menunjukkan metode HTTP tidak diizinkan.
6. 409 Conflict, menunjukkan adanya konflik data, misalnya username duplikat.
7. 500 Internal Server Error, menunjukkan terjadinya kesalahan internal server.

2.6 PHP Data Objects (PDO)

PHP Data Objects (PDO) merupakan antarmuka PHP untuk mengakses database. PDO mendukung penggunaan prepared statement sehingga parameter dapat dikirimkan secara terpisah dari perintah SQL. Mekanisme tersebut membantu mengurangi risiko SQL injection apabila diterapkan dengan benar.

2.7 Authentication dan Password Hashing

Authentication merupakan proses verifikasi identitas pengguna sebelum diberikan akses ke suatu sistem. Salah satu bagian penting dalam sistem authentication adalah pengelolaan password secara aman.

Password tidak seharusnya disimpan dalam bentuk teks biasa. PHP menyediakan fungsi password_hash() untuk menghasilkan hash password. Hash tersebut dapat diverifikasi ketika proses login menggunakan password_verify().

Pada praktikum ini, implementasi difokuskan pada registrasi pengguna dan penyimpanan password dalam bentuk hash. Proses login belum menjadi bagian implementasi utama.

BAB 3. IMPLEMENTASI DAN PENGUJIAN

3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini meliputi:

1. Sistem operasi Windows.
2. Laragon sebagai lingkungan server lokal.
3. Apache sebagai web server.
4. MySQL sebagai sistem manajemen database.
5. PHP sebagai bahasa pemrograman backend.
6. Visual Studio Code sebagai editor kode.
7. PowerShell sebagai alat pengujian request API.
8. Browser untuk memeriksa respons API.

3.2 Pembuatan Struktur Folder Proyek

Proyek praktikum dibuat pada direktori C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA 19 Postman dan API Authentication.

Struktur folder dibuat untuk memisahkan konfigurasi database, controller, model, komponen inti, file database, dan direktori publik.

Struktur folder proyek adalah sebagai berikut:

ACARA 19 Postman dan API Authentication

```
├── app
│   ├── config
│   │   └── Database.php
│   ├── controllers
│   │   ├── AuthController.php
│   │   └── MahasiswaController.php
│   ├── core
│   │   ├── autoload.php
│   │   ├── Response.php
│   │   └── Router.php
```

```
│   └── models
│       ├── Mahasiswa.php
│       └── User.php
├── database
│   └── database.sql
└── public
    ├── .htaccess
    └── index.php
```

Pemisahan file tersebut menerapkan pembagian tanggung jawab agar kode lebih terstruktur dan mudah dikelola.

3.3 Pembuatan Database

Database yang digunakan dalam praktikum ini bernama acara 19. Database tersebut digunakan untuk menyimpan data mahasiswa dan data pengguna yang melakukan registrasi.

Tabel users digunakan untuk menyimpan identitas pengguna dengan kolom id, username, email, password, role, dan created_at. Kolom username dan email diberi batasan unik agar tidak terdapat akun dengan username atau email yang sama.

Selain tabel users, tabel mahasiswa digunakan untuk menyimpan data mahasiswa dengan kolom id, nim, nama, prodi, dan created_at.

Pembuatan tabel dilakukan melalui phpMyAdmin menggunakan perintah SQL. Setelah tabel dibuat, data mahasiswa dimasukkan sebagai data pengujian endpoint.

3.4 Konfigurasi Koneksi Database

File app/config/Database.php digunakan untuk mengatur koneksi PHP dengan database MySQL. Koneksi dibuat menggunakan PDO dengan konfigurasi host, nama database, username, dan password database.

Pengaturan PDO mencakup penggunaan mode exception untuk menangani kesalahan database, pengaturan hasil query dalam bentuk associative array, serta penonaktifan emulasi prepared statement.

Dengan konfigurasi tersebut, model dapat mengakses data yang dibutuhkan oleh aplikasi melalui koneksi database yang telah disediakan.

3.5 Implementasi Komponen Inti dan Routing

Komponen inti aplikasi terdiri atas autoload.php, Response.php, dan Router.php.

File autoload.php digunakan untuk memuat class PHP secara otomatis berdasarkan lokasi file class. File Response.php digunakan untuk menghasilkan respons JSON dan mengatur HTTP status code.

Sementara itu, Router.php berfungsi menentukan controller yang dijalankan berdasarkan endpoint dan metode HTTP yang diterima. Endpoint mahasiswa digunakan untuk mengambil data mahasiswa, sedangkan endpoint register digunakan untuk proses registrasi pengguna.

File public/.htaccess mengatur URL rewriting agar request endpoint diteruskan ke public/index.php. File index.php menjadi pintu masuk utama yang memuat autoload dan menjalankan router.

3.6 Implementasi Model Mahasiswa

File app/models/Mahasiswa.php berfungsi mengelola akses data mahasiswa pada database. Model menyediakan fungsi untuk mengambil seluruh data mahasiswa dan mencari mahasiswa berdasarkan ID.

Pengambilan data dilakukan menggunakan PDO. Untuk pencarian berdasarkan ID, digunakan prepared statement dengan parameter sehingga nilai ID tidak digabungkan secara langsung ke dalam perintah SQL.

Model tersebut digunakan oleh MahasiswaController.php untuk menyediakan data kepada klien dalam format JSON.

3.7 Implementasi Model User

File app/models/User.php digunakan untuk mengelola data pengguna. Model ini memiliki fungsi create() untuk menyimpan data pengguna baru serta fungsi findByUsername() untuk mencari pengguna berdasarkan username.

Proses penyimpanan menggunakan prepared statement PDO. Password yang dikirimkan ke fungsi create() terlebih dahulu diubah menjadi hash oleh controller, sehingga password asli tidak disimpan sebagai teks biasa di database.

3.8 Implementasi Registrasi Pengguna

File app/controllers/AuthController.php digunakan untuk menangani proses registrasi melalui endpoint /register.

Controller membaca request body menggunakan php://input, kemudian mengubah data JSON menjadi array PHP menggunakan json_decode(). Data yang diterima terdiri atas username, email, dan password.

Sebelum disimpan, sistem melakukan pemeriksaan terhadap field wajib dan format email. Password kemudian diproses menggunakan password_hash() dengan algoritma default PHP. Apabila registrasi berhasil, API mengirimkan respons dengan status HTTP 201 Created.

Jika terjadi konflik username atau email, sistem mengembalikan status 409 Conflict. Kesalahan validasi input ditangani menggunakan status 400 Bad Request.

3.9 Pengujian Endpoint GET Mahasiswa

Pengujian endpoint mahasiswa dilakukan melalui browser dengan mengakses URL endpoint /mahasiswa. Pengujian ini bertujuan memastikan bahwa routing, koneksi database, model, dan controller dapat bekerja dengan benar.

Hasil pengujian menunjukkan bahwa API mengembalikan respons JSON dengan nilai success bernilai true, pesan keberhasilan, serta daftar data mahasiswa yang tersimpan dalam database.

Respons tersebut menunjukkan bahwa endpoint GET mahasiswa dapat mengambil dan menampilkan data dari database dengan benar.

```
{
  "success": true,
  "message": "Data mahasiswa berhasil diambil",
  "data": [
    {
      "id": 1,
      "nim": "260190001",
      "nama": "Budi Santoso",
      "prodi": "Teknik Informatika",
      "created_at": "2026-10-21 21:20:18"
    },
    {
      "id": 2,
      "nim": "260190002",
      "nama": "Andi Pratama",
      "prodi": "Manajemen Informatika",
      "created_at": "2026-10-21 21:20:18"
    },
    {
      "id": 3,
      "nim": "260190003",
      "nama": "Siti Aminah",
      "prodi": "Teknik Informatika",
      "created_at": "2026-10-21 21:20:18"
    }
  ]
}
```


Gambar 3.3 Hasil pengujian endpoint GET mahasiswa.

3.10 Pengujian Registrasi Pengguna

Pengujian registrasi dilakukan menggunakan PowerShell dengan metode POST dan header Content-Type: application/json. Data registrasi dikirim dalam bentuk JSON yang memuat username, email, dan password.

Pengujian registrasi dengan data valid menghasilkan respons yang menunjukkan bahwa akun berhasil dibuat. Respons mencakup ID pengguna, username, dan email, tanpa mengembalikan password kepada klien.

Hasil pengujian tersebut menunjukkan bahwa endpoint registrasi dapat menerima data JSON, memproses input, dan menyimpan pengguna baru ke database.



Gambar 3.4 Hasil pengujian registrasi pengguna.

3.11 Pengujian Validasi Input

Pengujian validasi dilakukan dengan mengirimkan request yang tidak memenuhi persyaratan. Skenario pertama menggunakan password kosong, sedangkan skenario kedua menggunakan format email yang tidak valid.

Hasil pengujian menunjukkan bahwa kedua request ditolak oleh API dengan status HTTP 400 Bad Request. Hal tersebut menunjukkan bahwa validasi input dijalankan sebelum data disimpan ke database.

Validasi tersebut membantu mencegah pembuatan akun dengan data wajib yang tidak lengkap atau format email yang tidak sesuai.



Gambar 3.5 Hasil pengujian validasi input.

3.12 Pengujian Username Duplikat

Pengujian username duplikat dilakukan dengan mengirimkan request registrasi menggunakan username faul_test, yang sebelumnya sudah berhasil didaftarkan.

Hasil pengujian menghasilkan status HTTP 409 Conflict. Status tersebut menunjukkan bahwa API menolak pembuatan akun dengan username yang sudah digunakan. Pemeriksaan ini didukung oleh batasan unik pada kolom username di database.

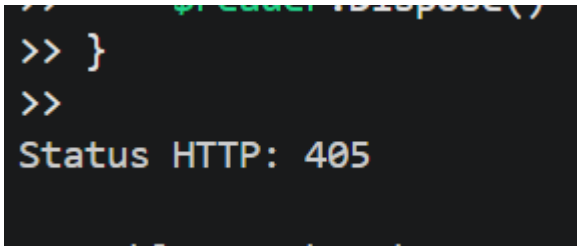
Pengujian tersebut menunjukkan bahwa sistem mampu menangani konflik data ketika terjadi registrasi dengan username yang sama.

3.13 Pengujian HTTP Method dan Endpoint Tidak Dikenal

Pengujian HTTP method dilakukan dengan mengakses endpoint /register menggunakan metode GET. Karena endpoint registrasi hanya menerima metode POST, request tersebut ditolak dengan status HTTP 405 Method Not Allowed.

Selain itu, pengujian dilakukan terhadap endpoint yang tidak terdaftar, yaitu /endpoint-salah. Hasil pengujian menunjukkan status HTTP 404 Not Found.

Kedua hasil tersebut menunjukkan bahwa router dapat membatasi metode HTTP sesuai ketentuan endpoint serta menangani request yang menuju endpoint yang tidak tersedia.



Gambar 3.6 Hasil pengujian HTTP method dan endpoint tidak dikenal.

3.14 Verifikasi Password Hashing

Verifikasi password dilakukan dengan memeriksa data pengguna pada tabel users melalui phpMyAdmin. Hasil pemeriksaan menunjukkan bahwa kolom

password tidak menyimpan password asli, melainkan string hash yang diawali dengan $2y$10$.

Awalan tersebut menunjukkan format hash bcrypt yang dihasilkan oleh password_hash() pada konfigurasi PHP yang digunakan. Dengan demikian, implementasi penyimpanan password telah berjalan sesuai tujuan keamanan dasar.

Password hash tidak dapat digunakan sebagai pengganti password asli ketika login. Pada implementasi autentikasi selanjutnya, proses verifikasi password dapat dilakukan menggunakan fungsi password_verify().

BAB 4. HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Berdasarkan implementasi dan pengujian yang telah dilakukan, API Acara 19 berhasil dibuat menggunakan PHP Native, PDO, dan MySQL. API menyediakan endpoint GET mahasiswa serta endpoint POST registrasi pengguna.

Pengujian endpoint GET mahasiswa menghasilkan status HTTP 200 dan menampilkan data mahasiswa dalam format JSON. Registrasi dengan data valid menghasilkan status HTTP 201 dan menyimpan akun baru ke database.

Pengujian validasi input menghasilkan status HTTP 400 untuk field wajib yang kosong dan format email yang tidak valid. Pengujian username duplikat menghasilkan status HTTP 409. Request GET terhadap endpoint registrasi menghasilkan status HTTP 405, sedangkan endpoint yang tidak dikenal menghasilkan status HTTP 404.

Pemeriksaan database juga menunjukkan bahwa password disimpan dalam bentuk hash, bukan teks biasa.

4.2 Pembahasan

Hasil praktikum menunjukkan bahwa API dapat menerima request, memproses data, mengakses database, dan menghasilkan response JSON sesuai kondisi yang terjadi. Penerapan HTTP status code membantu membedakan keberhasilan request, kesalahan validasi, konflik data, dan request yang tidak sesuai.

Penggunaan PDO dan prepared statement membantu meningkatkan keamanan akses database. Validasi input memastikan bahwa data yang diproses memenuhi persyaratan dasar, sedangkan password hashing membantu melindungi password pengguna ketika data database diperiksa.

Pengujian dilakukan menggunakan PowerShell sebagai alternatif Postman. Metode ini tetap memungkinkan pengiriman request HTTP dengan metode, header, dan body JSON yang ditentukan. Namun, penggunaan PowerShell pada praktikum

ini tidak mencakup seluruh fitur antarmuka Postman, seperti pengelolaan collection dan environment secara visual.

Implementasi yang dilakukan berfokus pada registrasi dan pengujian endpoint. Fitur login, verifikasi token, dan autentikasi pengguna pada endpoint yang dilindungi belum diimplementasikan dalam praktikum ini.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum Acara 19, dapat disimpulkan bahwa API menggunakan PHP Native dapat dikembangkan untuk mengambil data mahasiswa dan melakukan registrasi pengguna melalui request HTTP dengan format JSON.

Validasi input, penggunaan HTTP status code, prepared statement PDO, dan password hashing merupakan bagian penting dalam membangun API yang terstruktur dan lebih aman. Hasil pengujian menunjukkan bahwa API mampu menangani registrasi berhasil, input tidak valid, username duplikat, metode HTTP yang tidak diizinkan, dan endpoint yang tidak dikenal.

Pengujian melalui PowerShell dapat digunakan sebagai alternatif untuk mengirimkan request HTTP dan memeriksa respons API, meskipun tidak menyediakan seluruh fitur visual yang terdapat pada Postman.

5.2 Saran

Pengembangan selanjutnya dapat mencakup implementasi login menggunakan password_verify(), autentikasi berbasis token, serta perlindungan endpoint yang memerlukan hak akses tertentu. Pengujian juga dapat dikembangkan menggunakan Postman agar request dapat dikelompokkan dalam collection dan dikelola melalui environment.

Selain itu, validasi input, penanganan kesalahan database, serta dokumentasi endpoint perlu terus ditingkatkan agar API menjadi lebih aman, mudah diuji, dan mudah dipelihara.